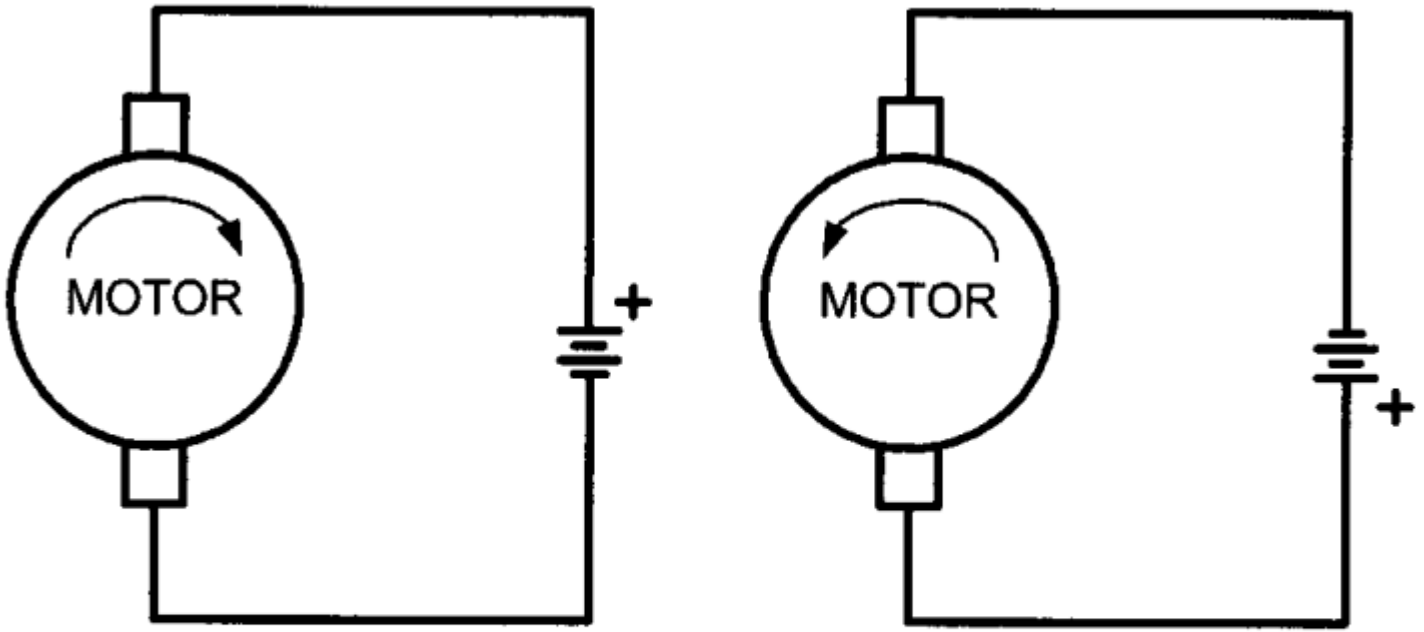
DC Motor Interfacing and Pulse Width Modulation (PWM)

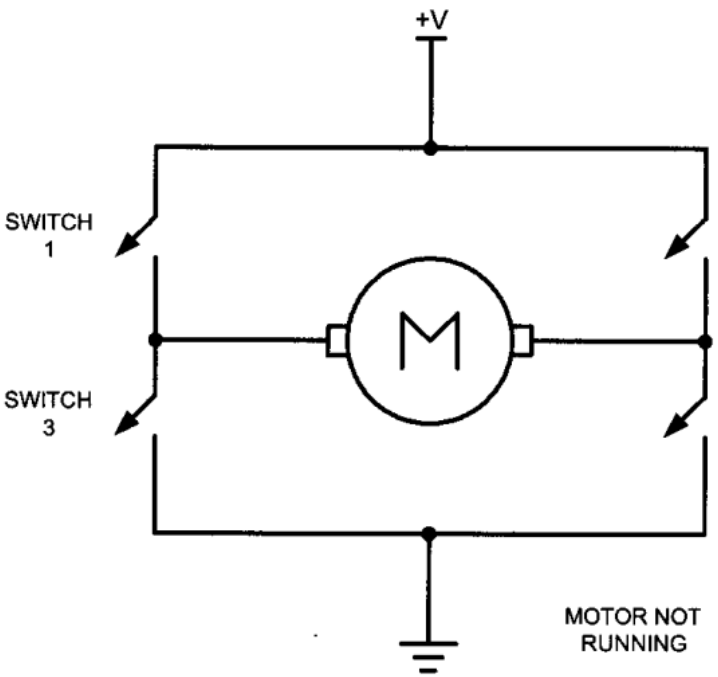
DC Motor:

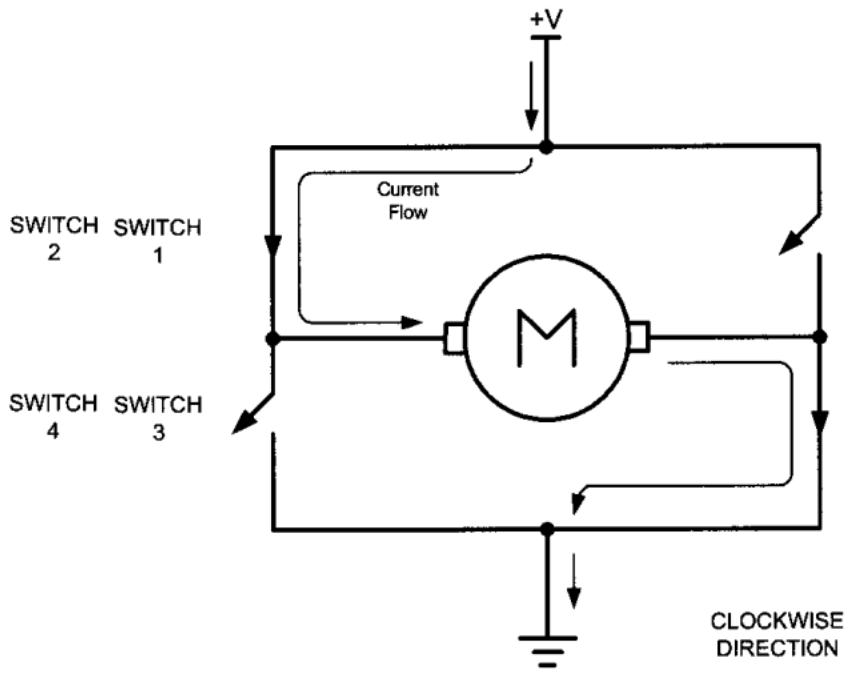


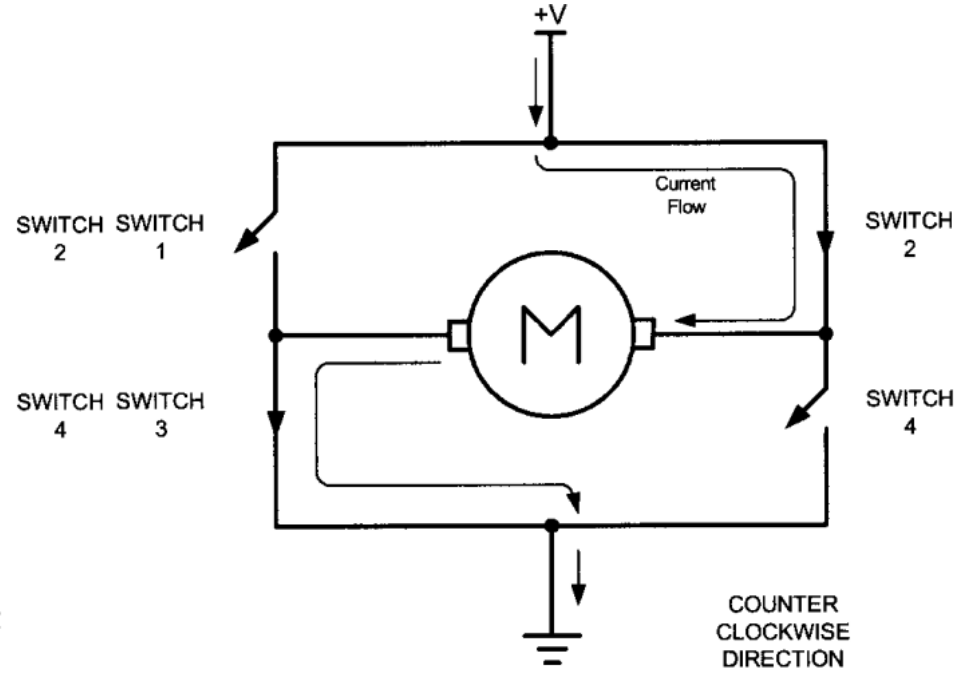
Clockwise Rotation

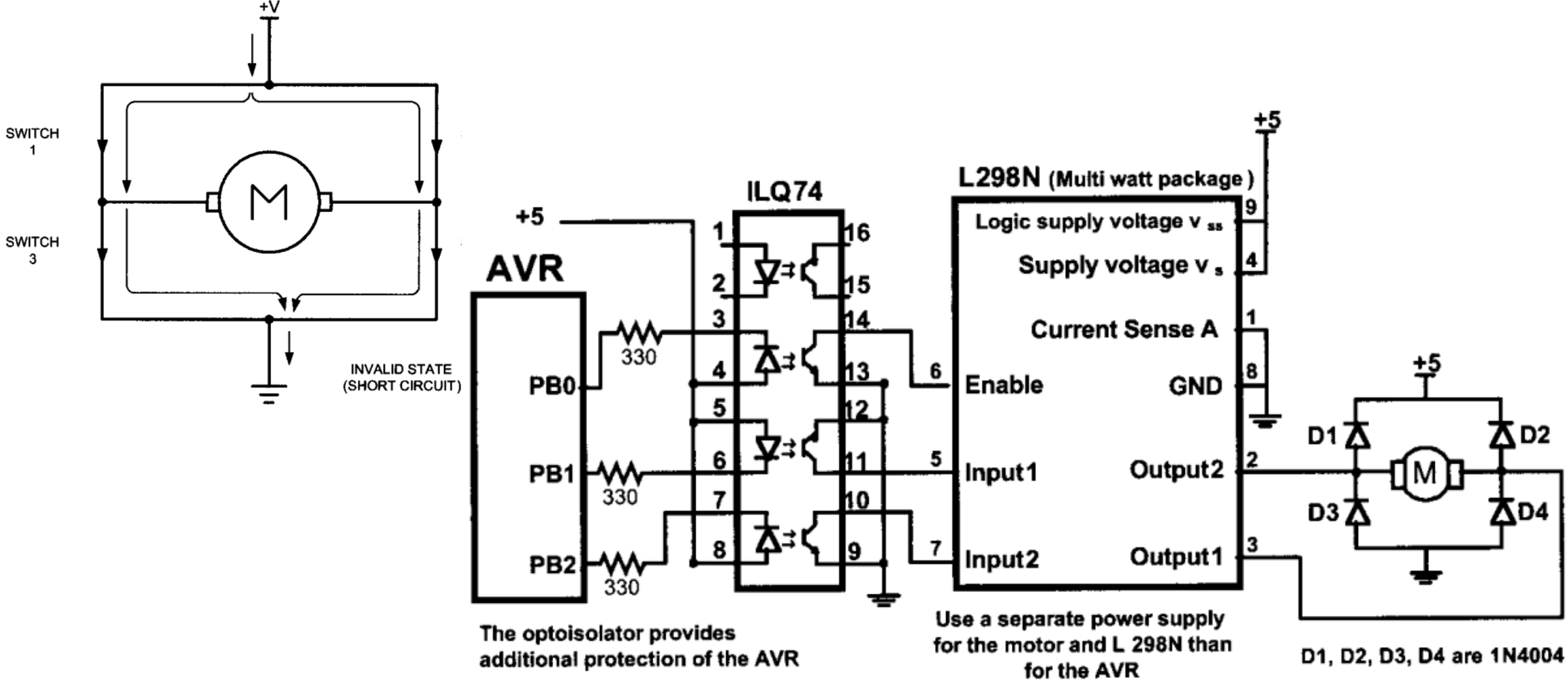
Counter-Clockwise Rotation

Part No.	Nominal Volts	Volt Range	Current	RPM	Torque
154915CP	3 V	1.5–3 V	0.070 A	5,200	4.0 g-cm
154923CP	3 V	1.5–3 V	0.240 A	16,000	8.3 g-cm
177498CP	4.5 V	3–14 V	0.150 A	10,300	33.3 g-cm
181411CP	5 V	3–14 V	0.470 A	10,000	18.8 g-cm

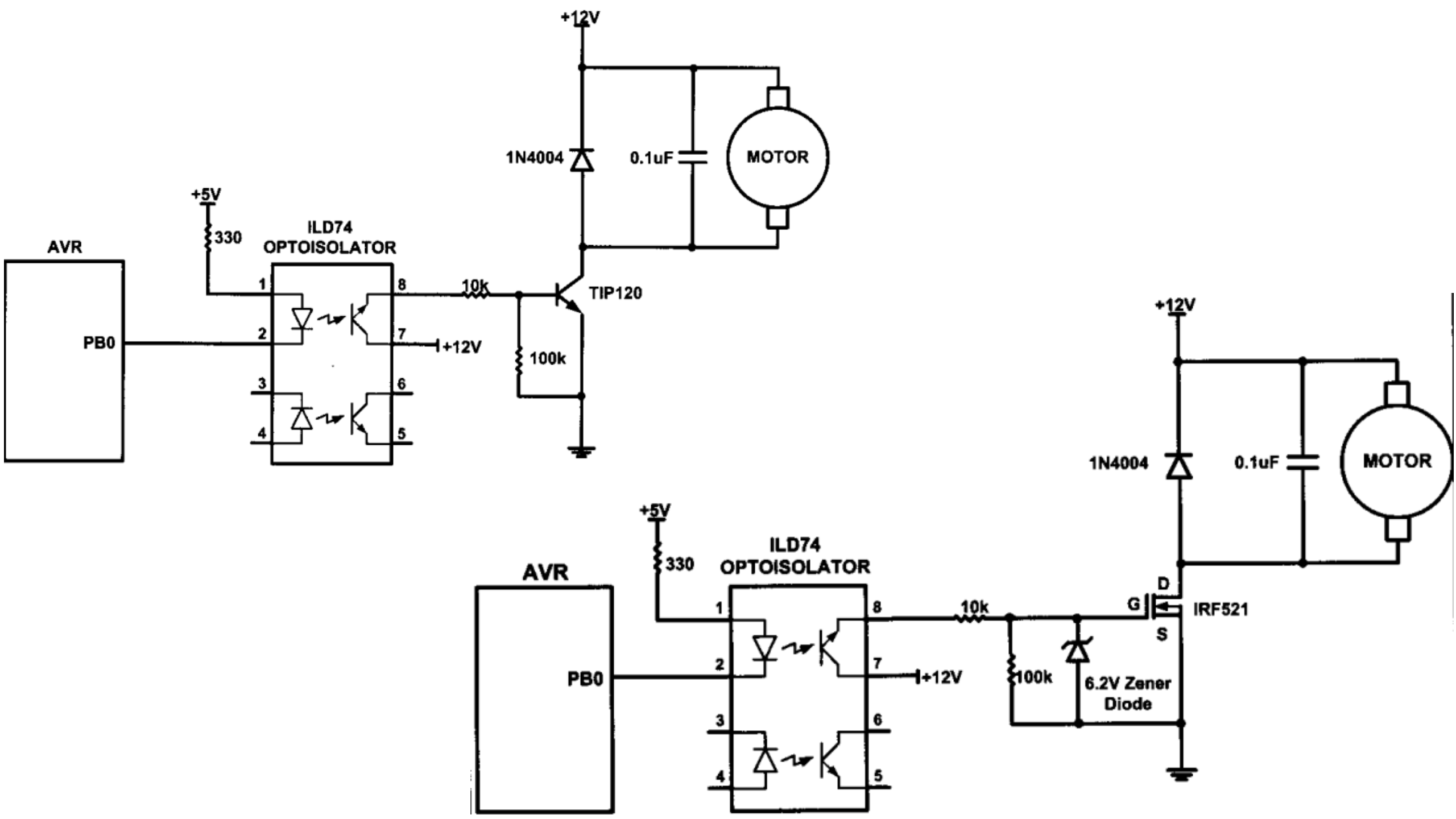


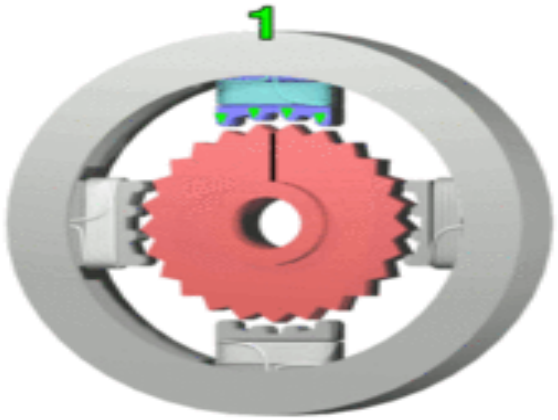




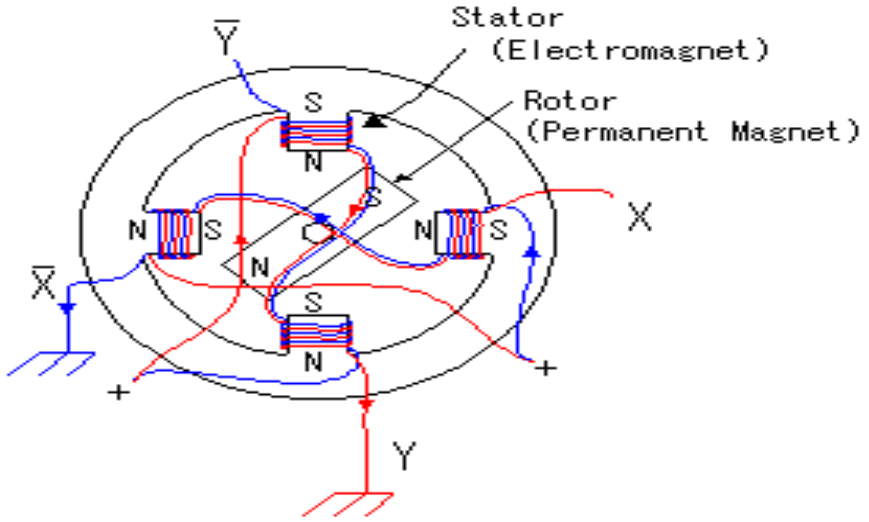


https://www.theengineeringprojects.com/2017/09/l298-motor-driver-library-proteus.html


+12V
1N4004
0.1uF
MOTOR
+5V
330
ILD74
OPTOISOLATOR
AVR
PB0
1
2
3
4
8
7
6
5
10k
+12V
TIP120
100k
+12V
1N4004
0.1uF
MOTOR
+5V
330
ILD74
OPTOISOLATOR
AVR
PB0
10k
D
G
S
IRF521
100k
6.2V Zener
Diode



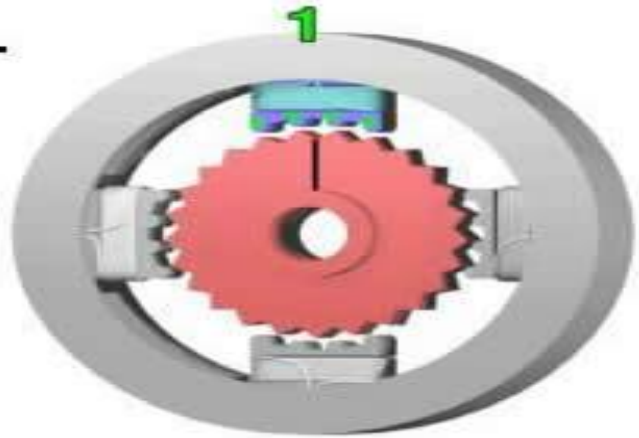
Stepper Motor



An electromagnetic actuator. It is an incremental drive (digital) actuator and is driven in fixed angular steps.

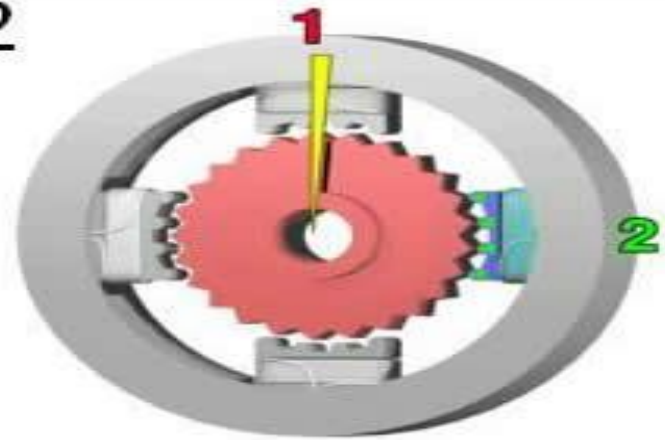
As for four poles, the top and the bottom and either side are a pair. X coil, X- coil and Y coil, Y-coil correspond respectively. For example, Y coil and Y- coil are put to the upper and lower pole. Y coil and Y- coil are rolled up for the direction of the pole to become opposite when applying an electric current to the X coil and applying an electric current to the X- coil. It is similar about and , too.

1



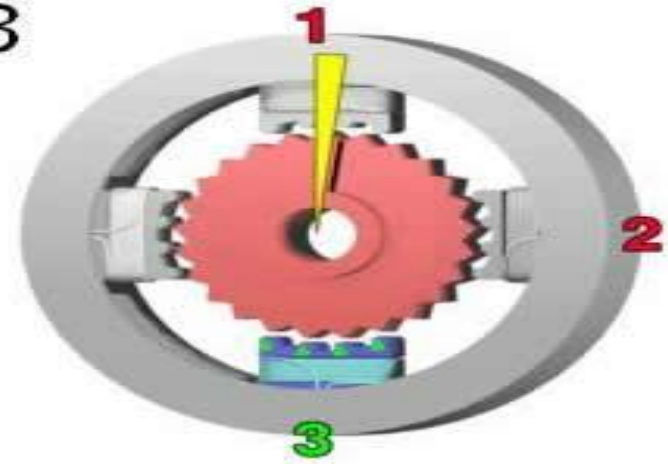
The top electromagnet (1) is turned on, attracting the nearest teeth of a gear-shaped iron rotor. With the teeth aligned to electromagnet 1, they will be slightly offset from electromagnet 2.

2



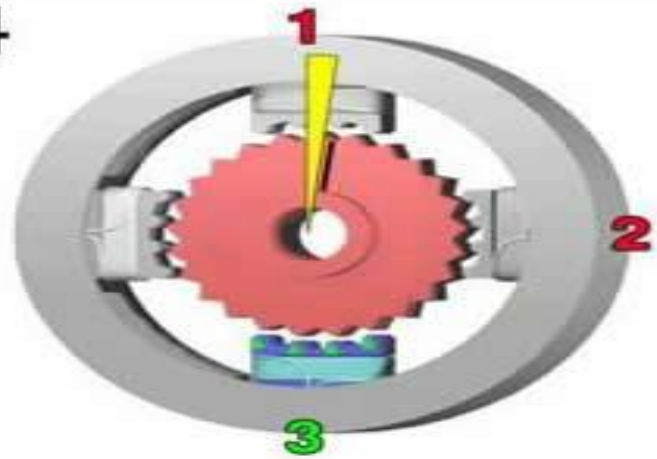
The top electromagnet (1) is turned off, and the right electromagnet (2) is energized, pulling the nearest teeth slightly to the right. This results in a rotation of 3.6° in this example.

3



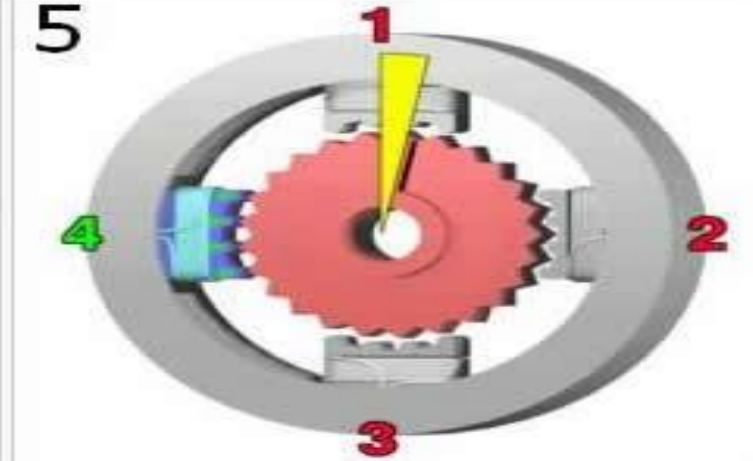
The bottom electromagnet (3) is energized; another 3.6° rotation occurs.

4

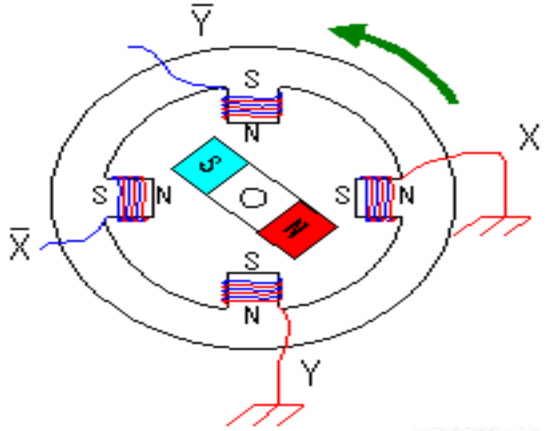


The bottom electromagnet (3) is energized; another 3.6° rotation occurs.

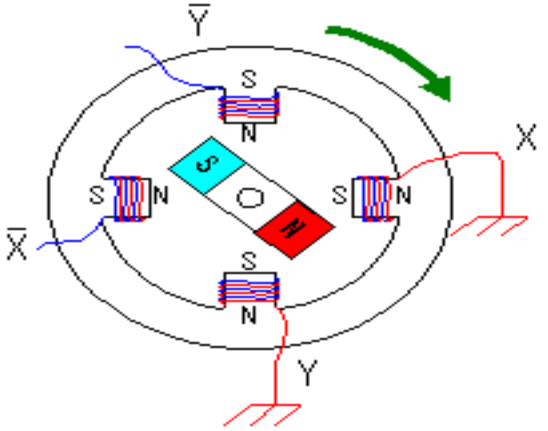
5



The left electromagnet (4) is enabled, rotating again by 3.6°. When the top electromagnet (1) is again enabled, the teeth in the sprocket will have rotated by one tooth position; since there are 25 teeth, it will take 100 steps to make a full rotation in this example.



X	$\overline{X}$	Y	$\overline{Y}$
0	1	0	1
0	1	1	0
1	0	1	0
1	0	0	1



X	$\overline{X}$	Y	$\overline{Y}$
0	1	0	1
1	0	0	1
1	0	1	0
0	1	1	0

$$\text{Step Angle}\,(\Theta_s) = \frac{360°}{S}$$

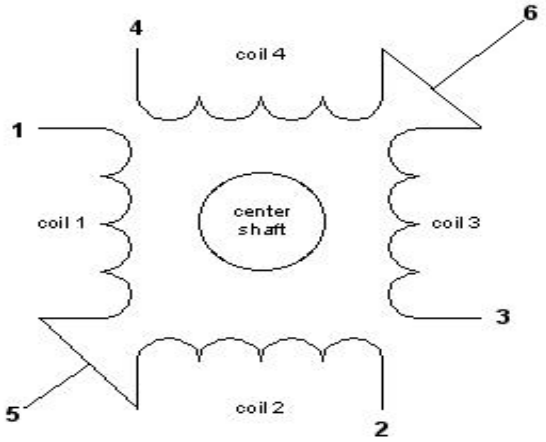
$$S = mN_r$$

$$m = number\ of\ phases$$

$$N_r = number\ of\ rotor\ teeth$$

Step Angle: minimum degree of rotation with a single step.
Steps Per Second: (RPM x Steps per revolution) / 60
No. of Teeth: Steps per revolution / 4
S: Steps per revolution

Unipolar stepper motor: The unipolar stepper motor has five or six wires and four coils (actually two coils divided by center connections on each coil). The center connections of the coils are tied together and used as the power connection.
Bipolar stepper motor: The bipolar stepper motor usually has four wires coming out of it. Unlike unipolar steppers, bipolar steppers have no common center connection. They have two independent sets of coils instead.



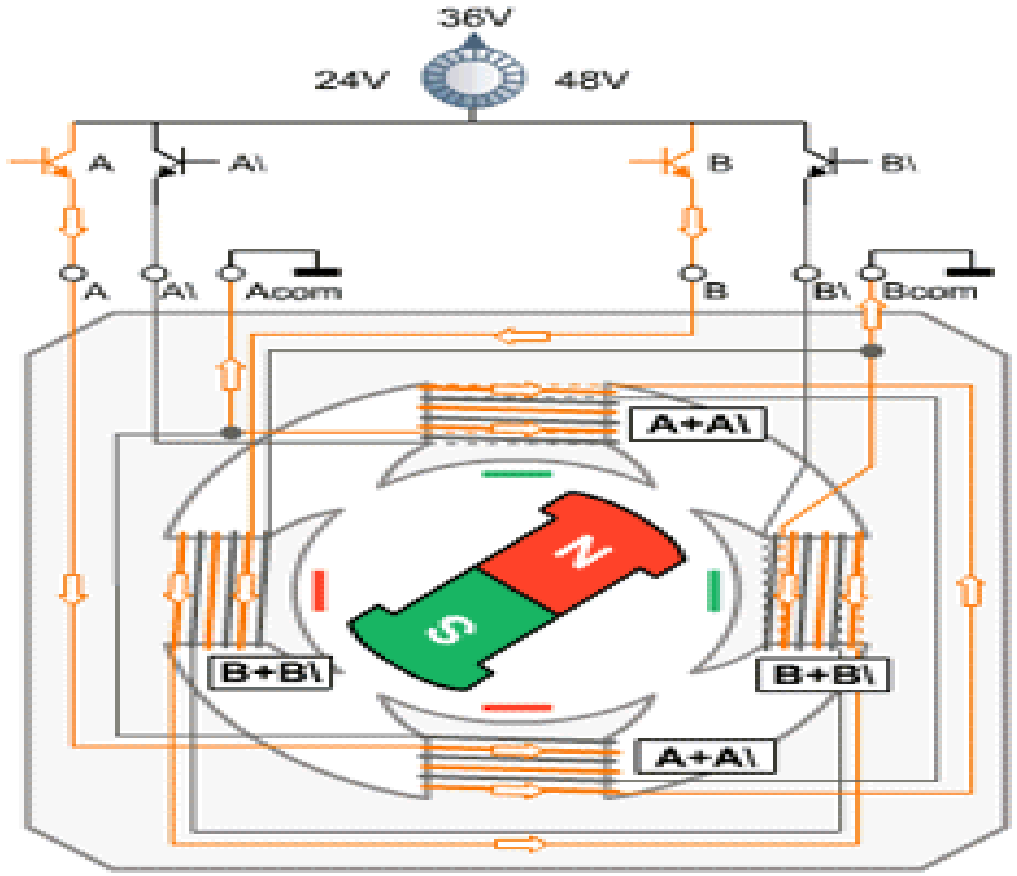
Stepper motors can be driven in two different patterns or sequences. namely,

1)Full Step Sequence

2)Half Step Sequence

Full Step Sequence: In the full step sequence, two coils are energized at the same time and motor shaft rotates. The order in which coils has to be energized is given in the table below.

Full Mode Sequence				
Step	A	B	A\	B\
0	1	1	0	0
1	0	1	1	0
2	0	0	1	1
3	1	0	0	1



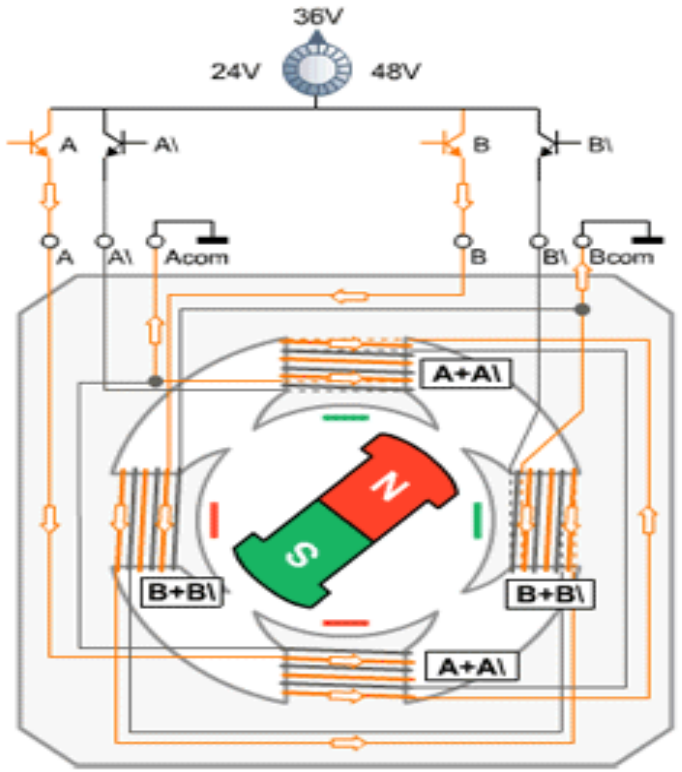
6 Lead Unipolar Driver

Unipolar control is the most simple and cost-effective way to drive a stepper motor, but results in approximately 30% less torque in comparison to the nowadays widely used bipolar drivers. Since the cost advantage is very small today due to cheap integrated circuits, bipolar drivers are now used in most new applications.

Stepmode								
F	0		1		2		3	
H	0	1	2	3	4	5	6	7
A	1	0	0	0	0	0	1	1
B	1	1	1	0	0	0	0	0
A\	0	0	1	1	1	0	0	0
B\	0	0	0	0	1	1	1	0
dez	12	4	6	2	3	1	9	8

Half Step Sequence: In Half mode step sequence, motor step angle reduces to half the angle in full mode. So the angualar resolution is also increased i.e. it becomes double the angular resolution in full mode. Also in half mode sequence the number of steps gets doubled as that of full mode. Half mode is usually preffered over full mode.

Half Mode Sequence				
Step	A	B	A\	B\
0	1	1	0	0
1	0	1	0	0
2	0	1	1	0
3	0	0	1	0
4	0	0	1	1
5	0	0	0	1
6	1	0	0	1
7	1	0	0	0



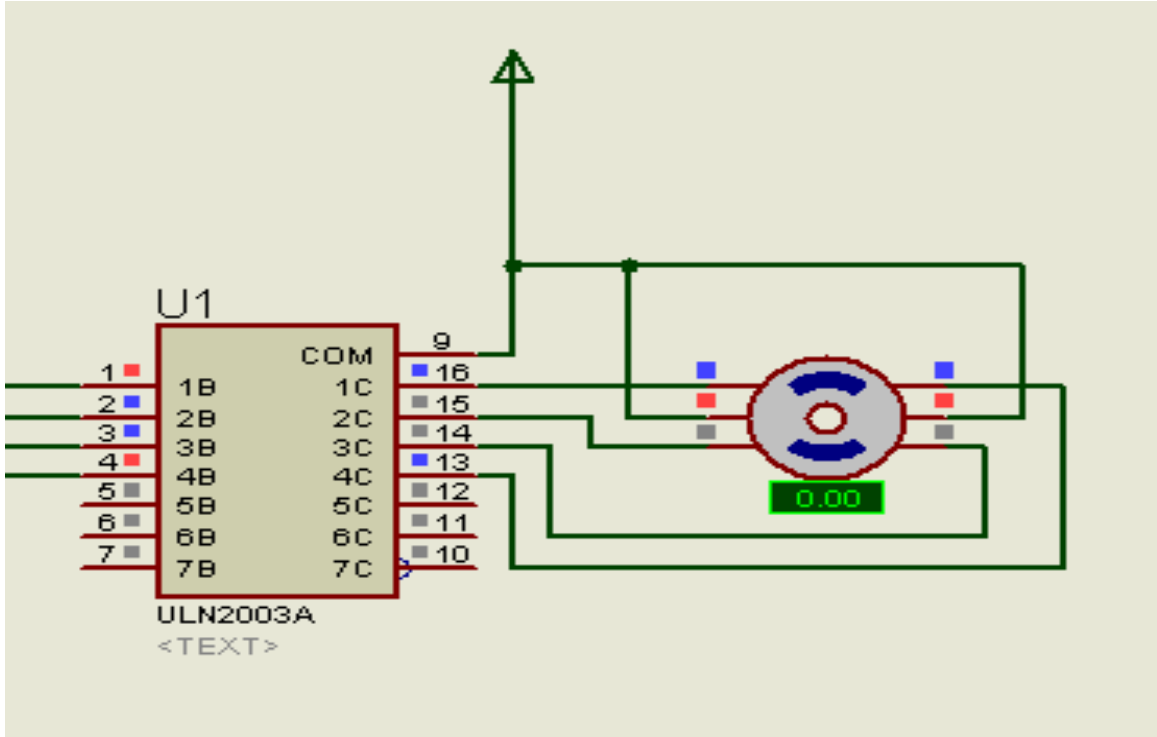
6 Lead Unipolar Driver

Unipolar control is the most simple and cost-effective way to drive a stepper motor, but results in approximately 30% less torque in comparison to the nowadays widely used bipolar drivers. Since the cost advantage is very small today due to cheap integrated circuits, bipolar drivers are now used in most new applications.

Stepmode								
F	0		1		2		3	
H	0	1	2	3	4	5	6	7
A	1	0	0	0	0	0	1	1
B	1	1	1	0	0	0	0	0
A\	0	0	1	1	1	0	0	0
B\	0	0	0	0	1	1	1	0
dez	12	4	6	2	3	1	9	8

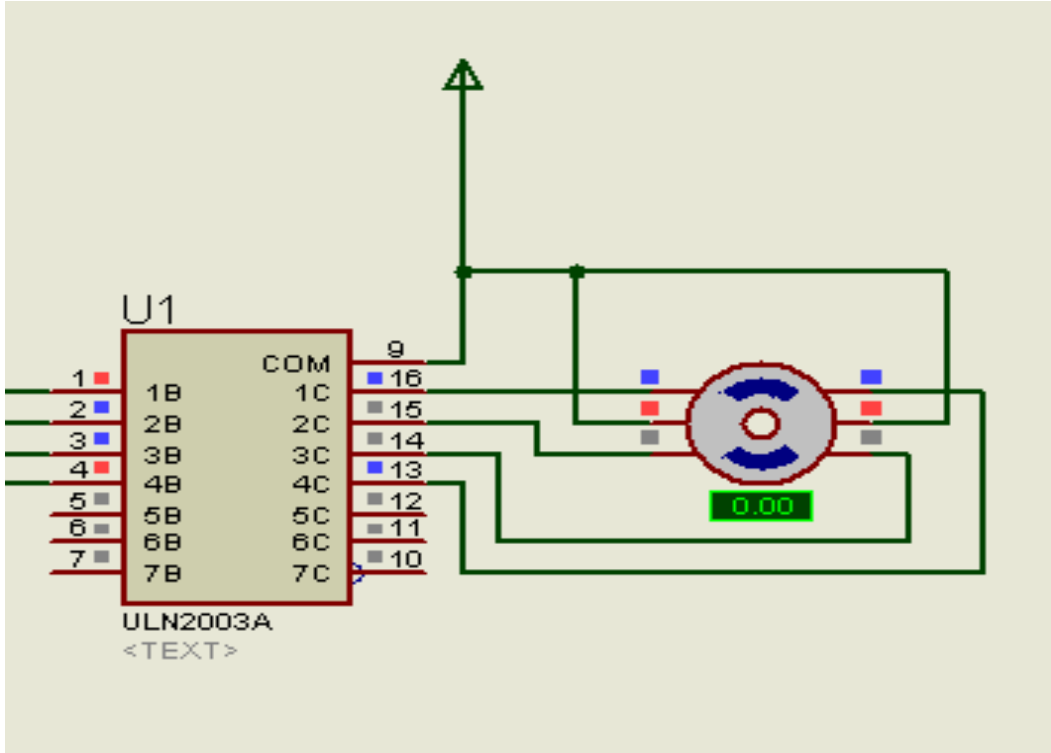
CODE:

```
#include <REG2051.H>.
#define stepper P1
void delay();

void main(){
        while(1){
                stepper = 0x0C;
                delay();
                stepper = 0x06;
                delay();
                stepper = 0x03;
                delay();
                stepper = 0x09;
                delay();
        }
}

void delay(){
        unsigned char i,j,k;
        for(i=0;i<6;i++)
                for(j=0;j<255;j++)
                        for(k=0;k<255;k++);
}
```

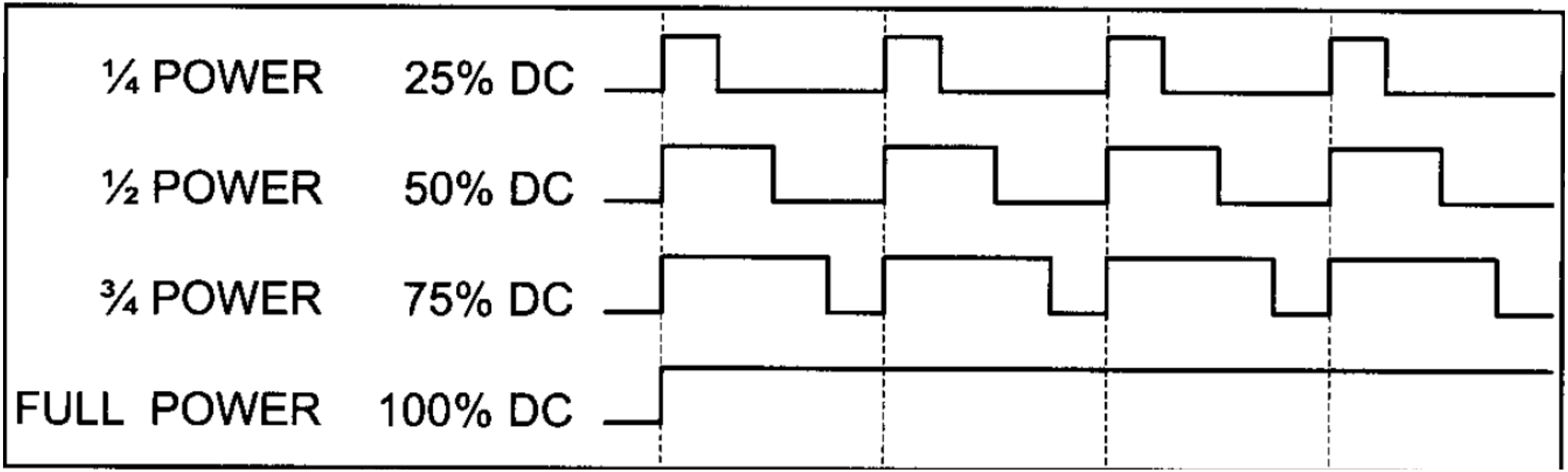


CODE:

```
void main(){
        while(1){
                stepper = 0x08;
                delay();
                stepper = 0x0C;
                delay();
                stepper = 0x04;
                delay();
                stepper = 0x06;
                delay();
                stepper = 0x02;
                delay();
                stepper = 0x03;
                delay();
                stepper = 0x01;
                delay();
                stepper = 0x09;
                delay();
        }
}
```



Pulse Width Modulation (PWM):



Datasheet Page: 115 for 8-bit PWM

Pulse Width Modulation (PWM) in Fast PWM Mode and Phase Correct PWM Mode:

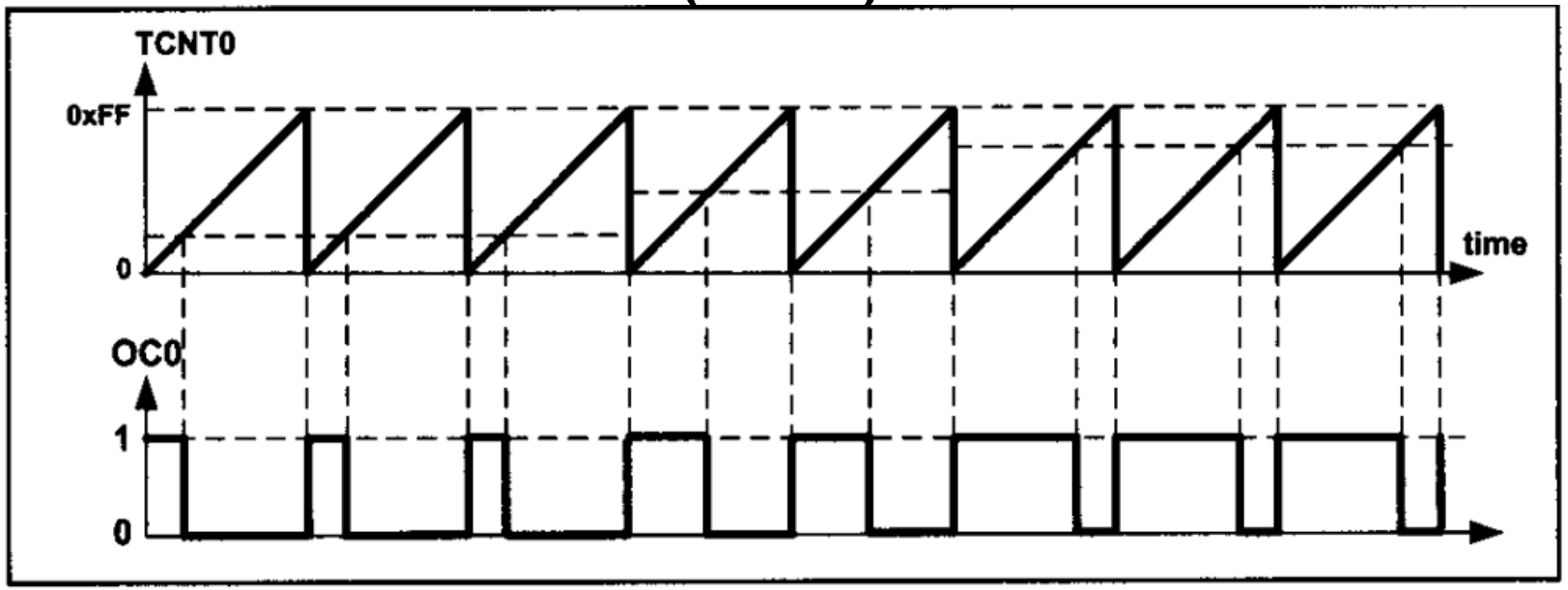


Figure 16-22. Fast PWM

$$Freq = \frac{Fc}{N * (1 + Top)} = \frac{Fc}{N * 256}$$

$$N\ (1, 8, 64, 256, 1024)$$

Non-Inverted Mode:

$$Duty\ Cycle = \frac{OCR0 + 1}{256} * 100$$

Inverted Mode:

$$Duty\ Cycle = \frac{255 - OCR0}{256} * 100$$

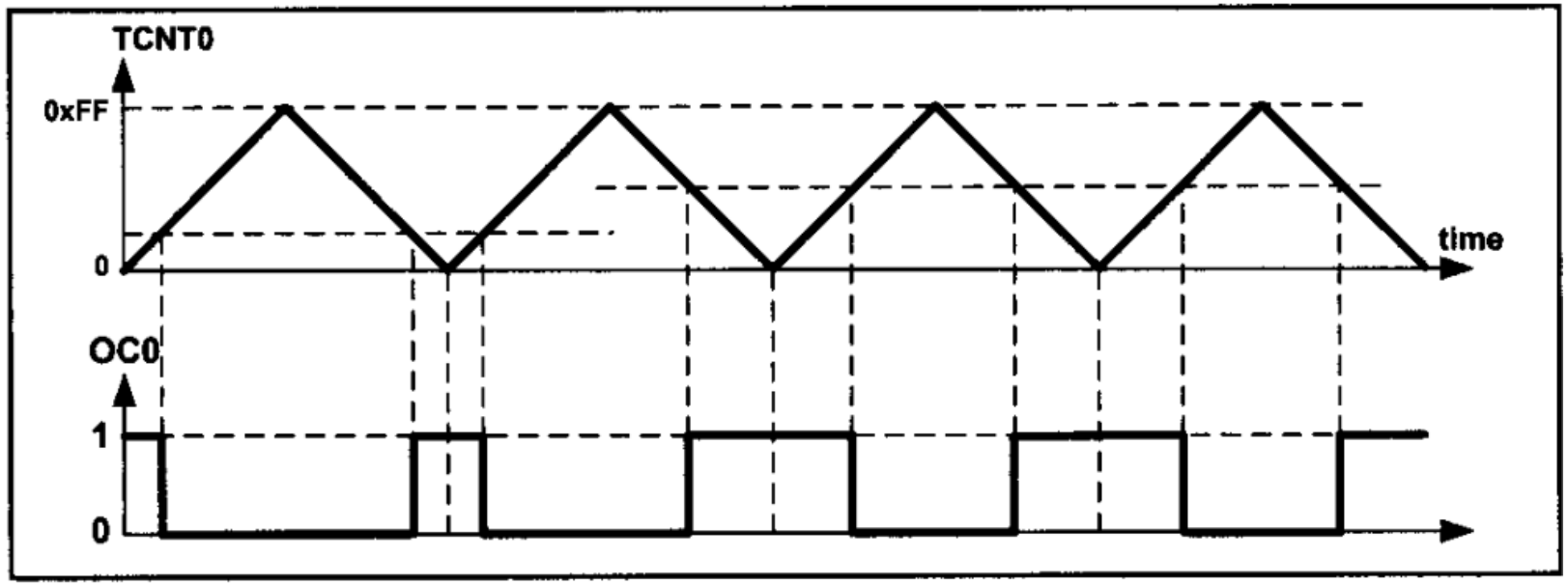


Figure 16-23. Phase Correct PWM

$$Freq = \frac{Fc}{N * 510}, N\ (1, 8, 64, 256, 1024)$$

Non-Inverted Mode:

$$Duty\ Cycle = \frac{OCR0}{255} * 100$$

Inverted Mode:

$$Duty\ Cycle = \frac{255 - OCR0}{255} * 100$$

Pulse Width Modulation (PWM) in Non-Inverted and Inverted Mode:

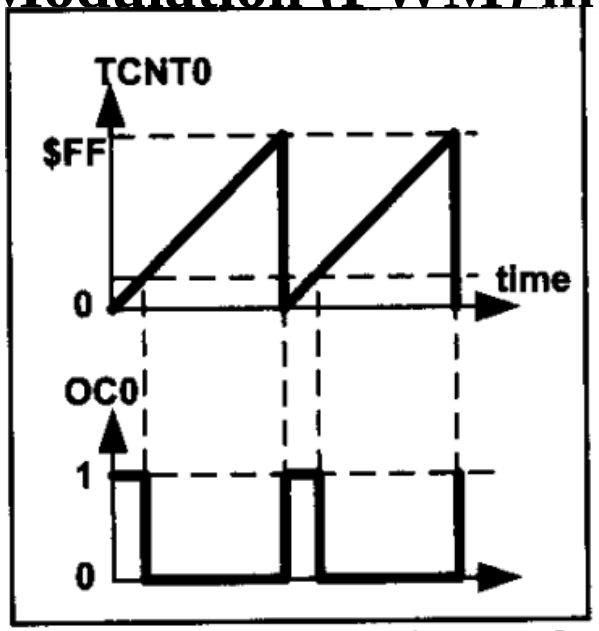


Figure 16-13A. Non-inverted

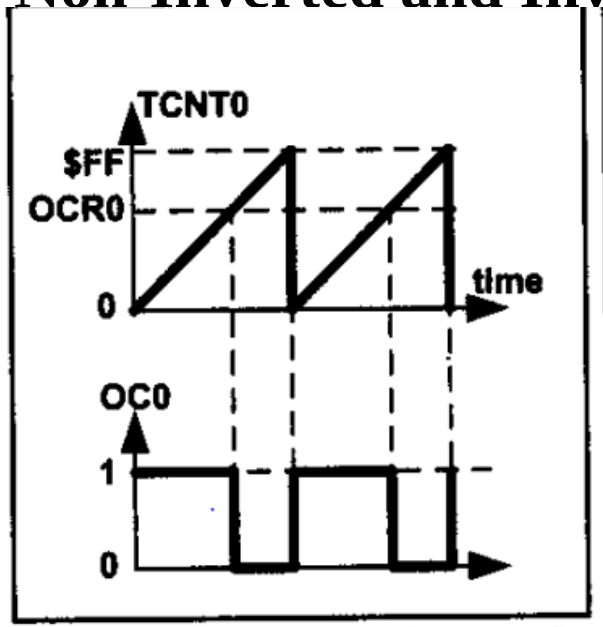


Figure 16-13B. Non-inverted

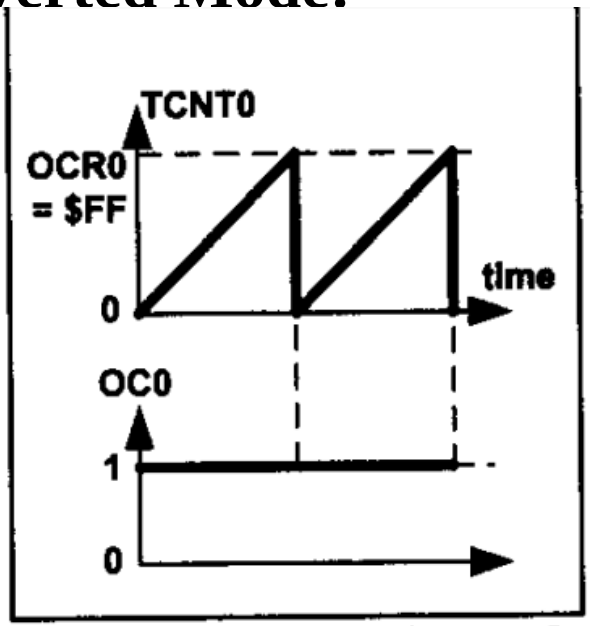


Figure 16-13C. Non-inverted

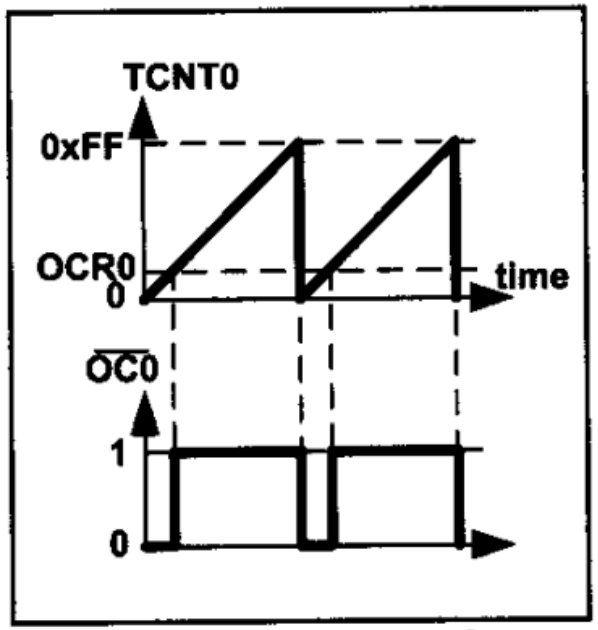


Figure 16-14A. Inverted

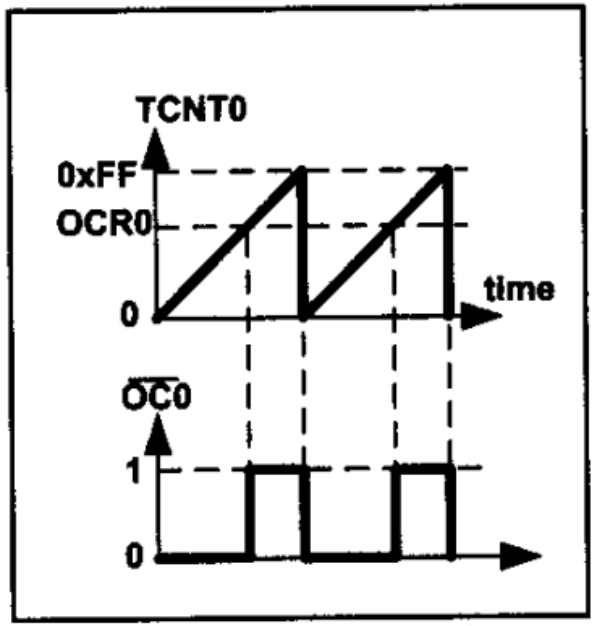


Figure 16-14B. Inverted

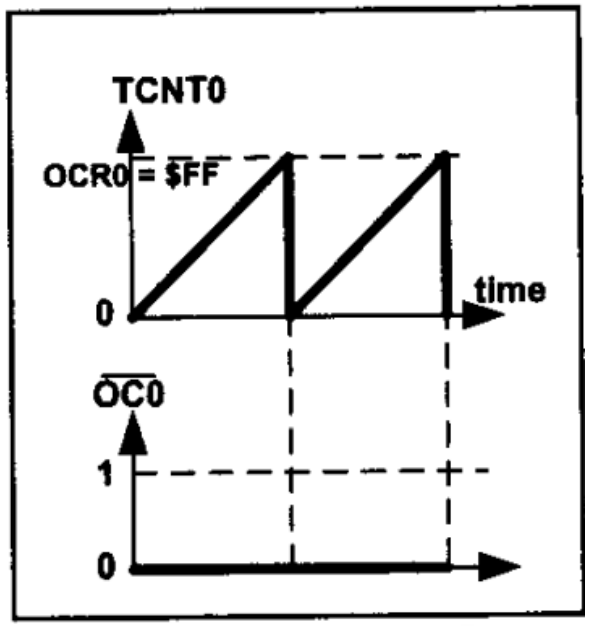


Figure 16-14C. Inverted

Pulse Width Modulation (PWM) in Phase Correct PWM Mode :

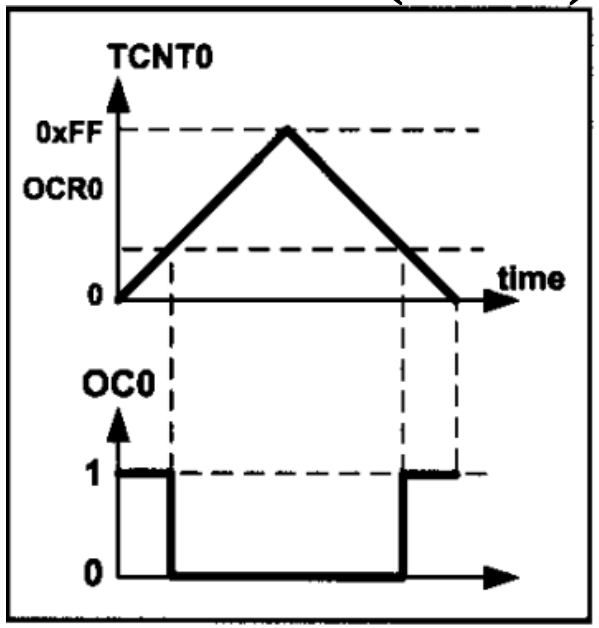


Figure 16-19A. Non-inverted

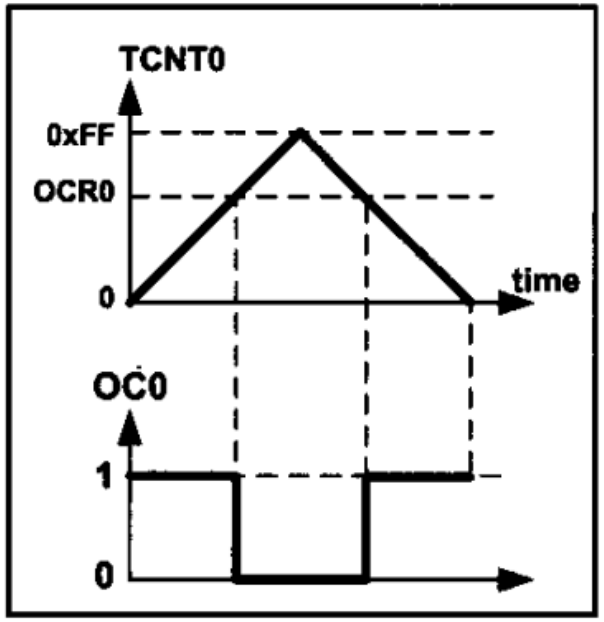


Figure 16-19B. Non-inverted

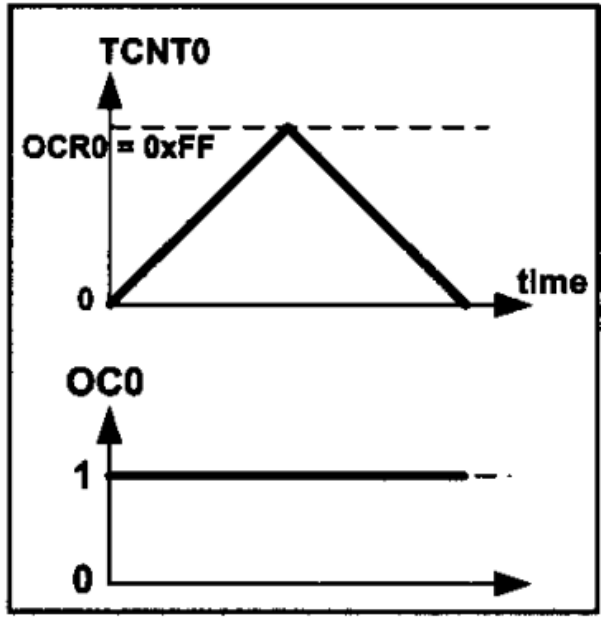


Figure 16-19C. Non-inverted

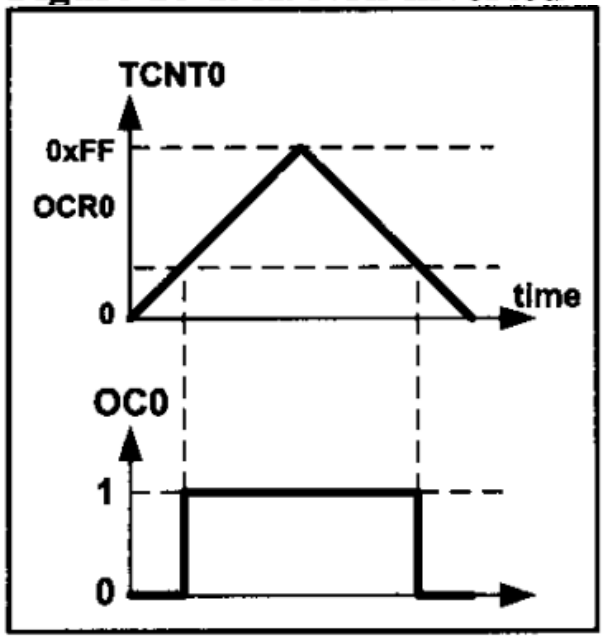


Figure 16-20A. Inverted

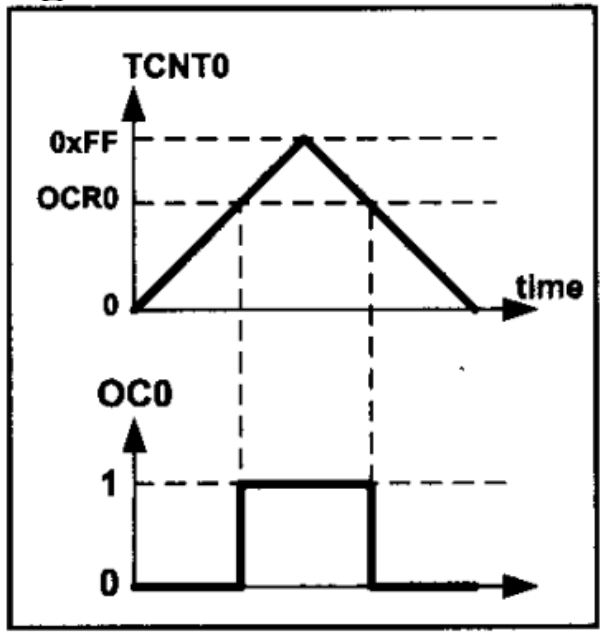


Figure 16-20B. Inverted

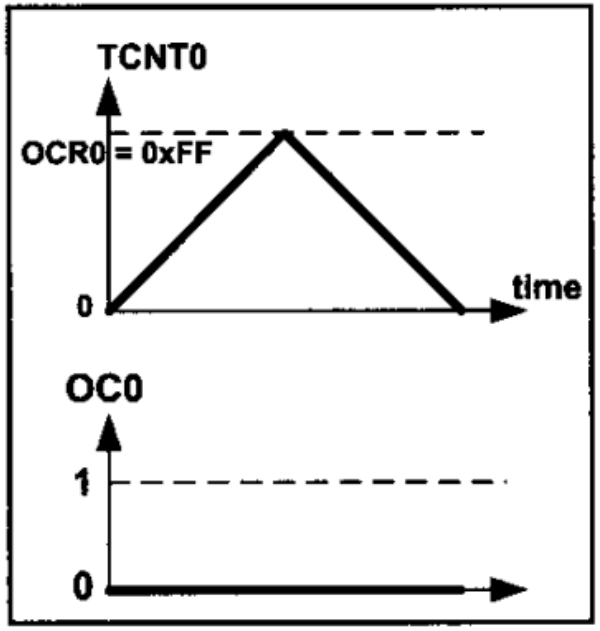


Figure 16-20C. Inverted

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:6 – COM0A1:0: Compare Match Output A Mode**

These bits control the Output Compare pin (OC0A) behavior. When OC0A (PB7) is connected to the pin, the function of the COM0A1:0 bits depends on the WGM02:0 bit setting.

Table 16-3. Compare Output Mode, Fast PWM Mode[1]

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected WGM02 = 1: Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match, set OC0A at BOTTOM (non-inverting mode)
1	1	Set OC0A on Compare Match, clear OC0A at BOTTOM (inverting mode)

Table 16-4. Compare Output Mode, Phase Correct PWM Mode[1]

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected WGM02 = 1: Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match when up-counting. Set OC0A on Compare Match when down-counting
1	1	Set OC0A on Compare Match when up-counting. Clear OC0A on Compare Match when down-counting

- **Bits 5:4 – COM0B1:0: Compare Match Output B Mode**

These bits control the Output Compare pin (OC0B) behavior. When OC0B (PG5) is connected to the pin, the function of the COM0B1:0 bits depends on the WGM02:0 bit setting.

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected
0	1	Toggle OC0B on Compare Match
1	0	Clear OC0B on Compare Match
1	1	Set OC0B on Compare Match

- **Bits 3, 2 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bits 1:0 – WGM01:0: Waveform Generation Mode**

Combined with the WGM02 bit found in the TCCR0B Register, these bits control the counting sequence of the counter, the source for maximum (TOP) counter value, and what type of waveform generation to be used. Modes of operation supported by the Timer/Counter unit are: Normal mode (counter), Clear Timer on Compare Match (CTC) mode, and two types of Pulse Width Modulation (PWM) modes.

Mode	WGM2	WGM1	WGM0	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on[1][2]
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	TOP	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

BOTTOM	The counter reaches the BOTTOM when it becomes 0x00.
MAX	The counter reaches its MAXimum when it becomes 0xFF (decimal 255).
TOP	The counter reaches the TOP when it becomes equal to the highest value in the count sequence. The TOP value can be assigned to be the fixed value 0xFF (MAX) or the value stored in the OCR0A Register. The assignment is dependent on the mode of operation.

TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	~~FOC0A~~	~~FOC0B~~	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – FOC0A: Force Output Compare A**

The FOC0A bit is only active when the WGM bits specify a non-PWM mode.

- **Bit 6 – FOC0B: Force Output Compare B**

The FOC0B bit is only active when the WGM bits specify a non-PWM mode.

- **Bits 5:4 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 3 – WGM02: Waveform Generation Mode**

(already explained in slide-6)

- **Bits 2:0 – CS02:0: Clock Select**

The three Clock Select bits select the clock source to be used by the Timer/Counter

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/(No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge
1	1	1	External clock source on T0 pin. Clock on rising edge

TCNT0 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
0x26 (0x46)	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Timer/Counter Register gives direct access, both for read and write operations, to the Timer/Counter unit 8-bit counter. Writing to the TCNT0 Register blocks (removes) the Compare Match on the following timer clock. Modifying the counter (TCNT0) while the counter is running, introduces a risk of missing a Compare Match between TCNT0 and the OCR0x Registers.

OCR0A – Output Compare Register A

Bit	7	6	5	4	3	2	1	0	
0x27 (0x47)	OCR0A[7:0]								OCR0A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Register A contains an 8-bit value that is continuously compared with the counter value (TCNT0). A match can be used to generate an Output Compare interrupt, or to generate a waveform output on the OC0A pin.

OCR0B – Output Compare Register B

Bit	7	6	5	4	3	2	1	0	
0x28 (0x48)	OCR0B[7:0]								OCR0B
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Output Compare Register B contains an 8-bit value that is continuously compared with the counter value (TCNT0). A match can be used to generate an Output Compare interrupt, or to generate a waveform output on the OC0B pin.

TIFR0 – Timer/Counter 0 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x15 (0x35)	–	–	–	–	–	OCF0B	OCF0A	TOV0	TIFR0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:3, 0 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 2 – OCF0B: Timer/Counter 0 Output Compare B Match Flag**

The OCF0B bit is set when a Compare Match occurs between the Timer/Counter and the data in OCR0B – Output Compare Register0 B. OCF0B is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF0B is cleared by writing a logic one to the flag. When the I-bit in SREG, OCIE0B (Timer/Counter Compare B Match Interrupt Enable), and OCF0B are set, the Timer/Counter Compare Match Interrupt is executed.

- **Bit 1 – OCF0A: Timer/Counter 0 Output Compare A Match Flag**

The OCF0A bit is set when a Compare Match occurs between the Timer/Counter0 and the data in OCR0A – Output Compare Register0. OCF0A is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, OCF0A is cleared by writing a logic one to the flag. When the I-bit in SREG, OCIE0A (Timer/Counter0 Compare Match Interrupt Enable), and OCF0A are set, the Timer/Counter0 Compare Match Interrupt is executed.

- **Bit 0 – TOV0: Timer/Counter0 Overflow Flag**

The bit TOV0 is set when an overflow occurs in Timer/Counter0. TOV0 is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, TOV0 is cleared by writing a logic one to the flag. When the SREG I-bit, TOIE0 (Timer/Counter0 Overflow Interrupt Enable), and TOV0 are set, the Timer/Counter0 Overflow interrupt is executed. The setting of this flag is dependent of the WGM02:0 bit setting.

TIMSK0 – Timer/Counter Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6E)	–	–	–	–	–	OCIE0B	OCIE0A	TOIE0	TIMSK0
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 7:3, 0 – Res: Reserved Bits**

These bits are reserved bits and will always read as zero.

- **Bit 2 – OCIE0B: Timer/Counter Output Compare Match B Interrupt Enable**

When the OCIE0B bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter Compare Match B interrupt is enabled. The corresponding interrupt is executed if a Compare Match in Timer/Counter occurs, that is, when the OCF0B bit is set in the Timer/Counter Interrupt Flag Register – TIFR0.

- **Bit 1 – OCIE0A: Timer/Counter0 Output Compare Match A Interrupt Enable**

When the OCIE0A bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter0 Compare Match A interrupt is enabled. The corresponding interrupt is executed if a Compare Match in Timer/Counter0 occurs, that is, when the OCF0A bit is set in the Timer/Counter 0 Interrupt Flag Register – TIFR0.

- **Bit 0 – TOIE0: Timer/Counter0 Overflow Interrupt Enable**

When the TOIE0 bit is written to one, and the I-bit in the Status Register is set, the Timer/Counter0 Overflow interrupt is enabled. The corresponding interrupt is executed if an overflow in Timer/Counter0 occurs, that is, when the TOV0 bit is set in the Timer/Counter 0 Interrupt Flag Register – TIFR0.

Assume XTAL = 16MHz, using non-inverted mode, write a program that generates a wave with frequency of 62500Hz and duty cycle of 75% using Fast PWM Mode.

$$Freq = \frac{Fc}{N * 256}, N\ (1, 8, 64, 256, 1024)$$

$$62500 = \frac{16M}{256 \times N} \Rightarrow N = 1, No\ Prescaler$$

Non-Inverted Mode:

$$Duty\ Cycle = \frac{OCR0 + 1}{256} * 100$$

$$75 = \frac{OCR0 + 1}{256} * 100 \Rightarrow OCR0 = 191$$

```
#include<avr/io.h>
int main()
{
  DDRB |= (1<<7);
  OCR0A = 191;
  TCCR0A = 0x83;
  TCCR0B = 0x09;
  while(1);
  return 0;
}
```

Inverted Mode:

$$Duty\ Cycle = \frac{255 - OCR0}{256} * 100$$

$$75 = \frac{255 - OCR0}{256} * 100 \Rightarrow OCR0 = 63$$

```
#include<avr/io.h>
int main()
{
  DDRB |= (1<<7);
  OCR0A = 63;
  TCCR0A = 0xC3;
  TCCR0B = 0x09;
  while(1);
  return 0;
}
```

Assume XTAL = 16MHz, using non-inverted mode, write a program that generates a wave with frequency of 7812.5Hz and duty cycle of 37.5% using Fast PWM Mode.

$$Freq = \frac{Fc}{N * 256}, N\ (1, 8, 64, 256, 1024)$$

$$7812.5 = \frac{16M}{256 \times N} \Rightarrow N = 8, 1/8\ Prescaler$$

Non-Inverted Mode:

$$Duty\ Cycle = \frac{OCR0 + 1}{256} * 100$$

$$37.5 = \frac{OCR0 + 1}{256} * 100 \Rightarrow OCR0 = 95$$

Inverted Mode:

$$Duty\ Cycle = \frac{255 - OCR0}{256} * 100$$

$$75 = \frac{255 - OCR0}{256} * 100 \Rightarrow OCR0 = 159$$

```
#include<avr/io.h>
int main()
{
  DDRB |= (1<<7);
  OCR0A = 95;
  TCCR0A = 0x83;
  TCCR0B = 0x0A;
  while(1);
  return 0;
}
```

```
#include<avr/io.h>
int main()
{
  DDRB |= (1<<7);
  OCR0A = 159;
  TCCR0 = 0xC3;
  TCCR0B = 0x0A;
  while(1);
  return 0;
}
```

Assume XTAL = 16MHz, using non-inverted mode, write a program that generates a wave with frequency of 31372.549Hz and duty cycle of 75% using Phase Correct PWM Mode.

$$Freq = \frac{Fc}{N * 510}, N\ (1, 8, 64, 256, 1024)$$

$$31372.549 = \frac{16M}{510 \times N} \Rightarrow N = 1, No\ Prescaler$$

Non-Inverted Mode:

$$Duty\ Cycle = \frac{OCR0}{255} * 100$$

$$75 = \frac{OCR0}{255} * 100 \Rightarrow OCR0 = 191$$

Inverted Mode:

$$Duty\ Cycle = \frac{255 - OCR0}{255} * 100$$

$$75 = \frac{255 - OCR0}{255} * 100 \Rightarrow OCR0 = 63$$

```
#include<avr/io.h>
int main()
{
  DDRB |= (1<<7);
  OCR0A = 191;
  TCCR0A = 0x81;
  TCCR0B = 0x09;
  while(1);
  return 0;
}
```

```
#include<avr/io.h>
int main()
{
  DDRB |= (1<<7);
  OCR0A = 63;
  TCCR0A = 0xC1;
  TCCR0B = 0x09;
  while(1);
  return 0;
}
```

Assume XTAL = 16MHz, using non-inverted mode, write a program that generates a wave with frequency of 61Hz and duty cycle of 87.5% using Phase Correct PWM Mode.

- **Non-Inverted Fast PWM Mode duty cycle changed between 1/256 and 256/256. It can not generate 0%.**
- **Inverted Fast PWM Mode duty cycle changed between 0/256 and 255/256. It can not generate 100%.**
- **Phase Correct PWM Mode duty cycle changed between 0/255 and 255/255. It can generate wave between 0% (completely OFF) and 100% (completely ON).**
- **For driving motors, Phase Correct PWM Mode is preferred over Fast PWM Mode.**
- **Fast PWM Mode frequency is 2x of the Phase Correct PWM Mode.**
- **Fast PWM Mode is preferred when high frequency wave is need to be generated.**